DUALE HOCHSCHULE SACHSEN

Verteilte Systeme

Wintersemester 2026

Dr. Michael Hofmann

DHSN: Glauchau

14.09.2026

Allgemeine Informationen

Ablauf

- 6 Termine mit je 2x2 LVS (2x90 Minuten) = 24 LVS

$\rightsquigarrow$ jeweils „Vorlesung“ + Beispiele und ggf. Übungsaufgaben

- Vorlesungsmaterialen $\rightsquigarrow$ OPAL: Theorie der Betriebssysteme und verteilten Systeme
- Abschlussklausur: 120 Minuten $\rightsquigarrow$ $\frac{1}{3}$ für Verteilte Systeme

Literatur

- Andrew S. Tanenbaum und Maarten van Steen. *Verteilte Systeme: Grundlagen und Paradigmen.* 2. Ausgabe, Addison Wesley, 2007.

Themenüberblick (aktueller Plan)

(1) 31.08.2026
- Einführung

(2) 10.09.2026
- Architekturen

(3) 14.09.2026
- Prozesse

(4) 14.10.2026
- Kommunikation

(5) 27.10.2026
- Synchronisation

(6) 02.11.2026
- Konsistenz und Replikation
- Fehlertoleranz
- Wiederholung, Fragen & Antworten

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Prozesse

Betriebssystemsicht

- Definition ⇝ *Programme in Ausführung*
- Herausforderungen ⇝ Verwaltung und Scheduling

Verteilte Systeme

- Threads ⇝ Überlagerung von Kommunikation und lokaler Verarbeitung
- Virtualisierung ⇝ Portabilität und Fehlerisolierung
- Clients ⇝ Nutzerschnittstellen und Verteilungstransparenz
- Server ⇝ Generelle Implementierungsaspekte
- Migration ⇝ Prozessen bzw. Code

Inhalt

3. Prozesse
- Threads
- Virtualisierung
- Client-Realisierungen
- Server-Realisierungen
- Code-Migration

Threads

Was der Unterschied zwischen Prozessen und Threads?

Threads

Was der Unterschied zwischen Prozessen und Threads?

Eigenschaften von Prozessen

- Vollständiger Zustand ⇝ CPU-Kontext, Speicher, Ressourcen, ...
- Prozesse sind isoliert voneinander
- Kommunikation ist aufwändig ⇝ Interprozesskommunikation (IPC)
- grobe Granularität ⇝ hoher Overhead

Eigenschaften von Threads

- Minimaler Zustand, d. h. nur CPU-Kontext
- Mehrere Threads innerhalb eines Prozesses
- Kommunikation über gemeinsamen Speicher ⇝ Synchronisation notwendig
- feine Granularität ⇝ hohe Performance

Threads in normalen (nicht-verteilten) Systemen

Welche Vorteile bieten Threads?

Threads in normalen (nicht-verteilten) Systemen

Welche Vorteile bieten Threads?

Überbrückung blockierender Operationen

- Trennung von grafischer Nutzerschnittstelle und Verarbeitung
- Überlagerung von blockierenden Systemaufrufen

Nutzung mehrere CPUs bzw. Kerne

- Aufteilung rechenintensiver Operationen auf mehrere Threads
- Parallele Ausführung von Threads auf CPUs bzw. Kernen

Aufteilung von „großen“ Anwendungen

- Kollektion von kooperierenden Programmen

⇝ Nutzung von Threads anstatt Prozessen mit IPC

Strukturierung von Implementierungen

- Ausführung von unabhängigen Aufgaben

Threads in verteilten Systemen (1)

- Nutzung blockierenden Systemaufrufe ohne den Prozess zu blockieren
- Verwaltung mehrerer logischer Verbindungen

Multi-threaded Clients

- Verteilte Systeme müssen lange Nachrichtenlaufzeiten verbergen
  ⇝ hunderte Millisekunden bis Sekunden
- Ansatz: Kommunikation starten und mit anderen Aufgaben fortfahren

Beispiel: Webbrowser

- Seite laden und anzeigen, während Bilder usw. parallel nachgeladen werden
- Threads holen Bilder usw. über separate Verbindungen

⇝ Threads nutzen blockierende Systemaufrufe, ohne den Prozess zu blockieren

\+ Nutzer können mit Seite interagieren, während andere Inhalte laden

\+ Lastverteilung auf replizierte Webserver durch parallele Verbindungen

Threads in verteilten Systemen (2)

- Hauptnutzung von Multithreading erfolgt in Servern

⇝ Vereinfachung der Server-Implementierungen

⇝ Ausnutzung von Parallelisierung zur Performanceverbesserung

Multi-threaded Server

- Server sollten mehrere Anfragen verarbeiten können
- Server müssen auf lokale Ressourcen warten können

Beispiel: Dateiserver

- Single-Threaded: Hauptschleife die Anfragen sequentiell abarbeitet
  - − Hohe Wartezeiten bei konkurrierenden Anfragen
  - − Inaktive CPU während Festplattenzugriffen
- Multi-Threaded: Dispatcher-Thread verteilt Anfragen auf Worker-Threads
  - \+ Parallele Abarbeitung konkurrierenden Anfragen
  - − Aktive CPU-Nutzung während Festplattenzugriffen

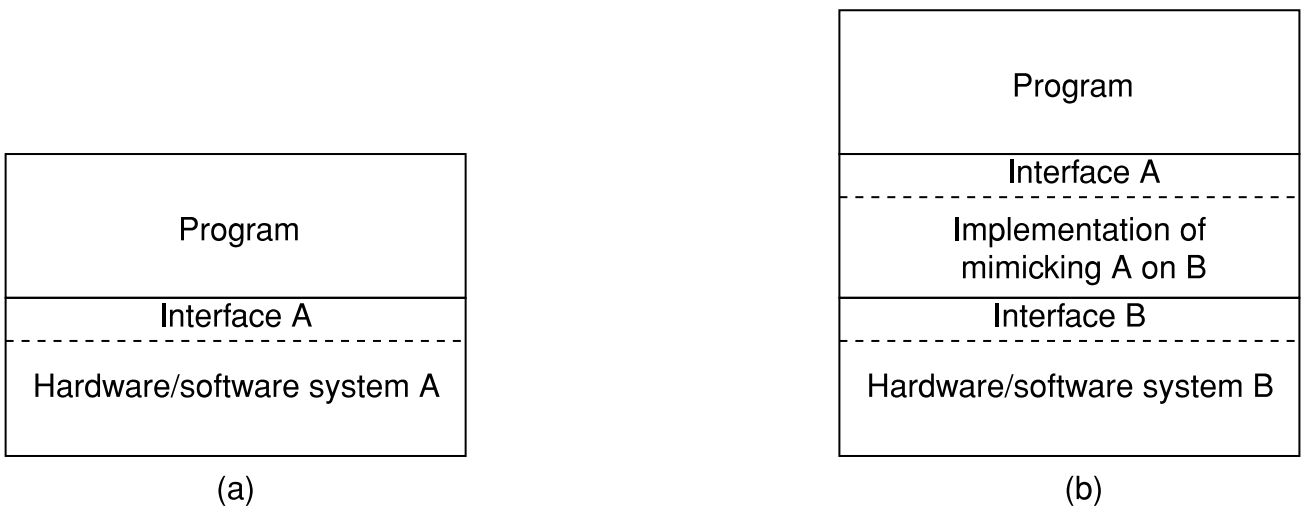
Inhalt

3. Prozesse
- Threads
- Virtualisierung
- Client-Realisierungen
- Server-Realisierungen
- Code-Migration

Virtualisierung

- Prozesse und Threads ermöglichen gemeinsame Nutzung einer CPU
- Virtualisierung erweitert das Konzept auf andere *Ressourcen*

Programmausführung ohne und mit Virtualisierung



(a) Direkte Ausführung auf System A

(b) Virtualisierte Ausführung in System A auf System B

Virtualisierung in verteilten Systemen

Frühe Entwicklung

- Unterstützung von Legacy-Software auf teurer Mainframes
- Günstige Hardware und wenig Betriebssysteme $\rightsquigarrow$ Rückgang der Relevanz

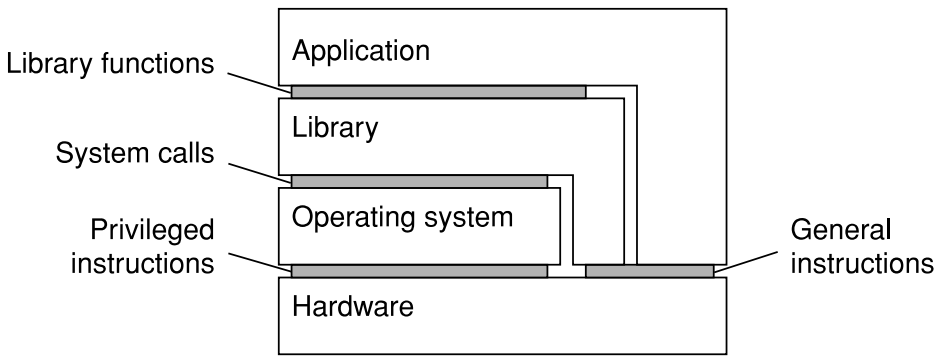
Ab 1990er Jahre

- Legacy-Unterstützung $\rightsquigarrow$ alte Anwendungen können auf neuen Plattformen ausgeführt werden
- Netzwerk- und Servervielfalt $\rightsquigarrow$ heterogene Systeme sind einfacher verwaltbar, da Anwendungen in eigenen VM-Umgebungen laufen können
- Portabilität & Flexibilität $\rightsquigarrow$ Komplette Systeme (inkl. Anwendungen und Betriebssystem) können leicht repliziert und migriert werden, z. B. für Content Delivery Networks (CDNs)

Architektur virtueller Maschinen (1)

- Virtualisierung erfolgt durch Nachahmung von Schnittstellen
- Computersysteme umfassen vier generelle Schnittstellen

Schnittstellen innerhalb eines Computersystems



1. Bibliotheksfunktionen (APIs) zw. Anwendungen und Bibliotheken
2. Systemaufrufe zw. Bibliotheken und Betriebssystem
3. Privilegierte Instruktionen zw. Betriebssystem und Hardware
4. Allgemeine Instruktionen zw. Anwendung und Hardware

- Virtualisierung $\rightsquigarrow$ Virtuelle Maschine (VM) stellt Schnittstelle bereit

Arten von Virtualisierung

Welche Arten von Virtualisierung gibt es?

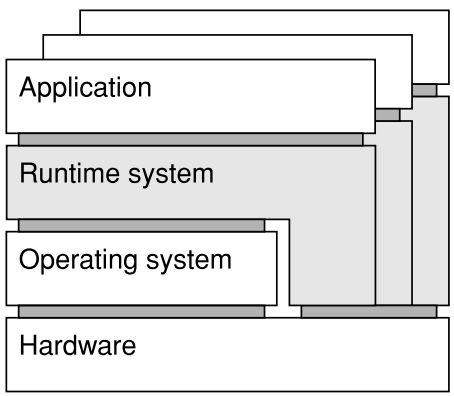
1. *Process Virtual Machine*
2. *Virtual Machine Monitor (VMM)*
3. *Containerization*

Arten von Virtualisierung

Welche Arten von Virtualisierung gibt es?

1. *Process Virtual Machine*
2. *Virtual Machine Monitor (VMM)*
3. *Containerization*

Process Virtual Machine



- Laufzeitsystem führt Anwendungen mit abstrahierten Instruktionen aus
- Instruktionen können vom Laufzeitsystem interpretiert oder emuliert werden

Beispiele:

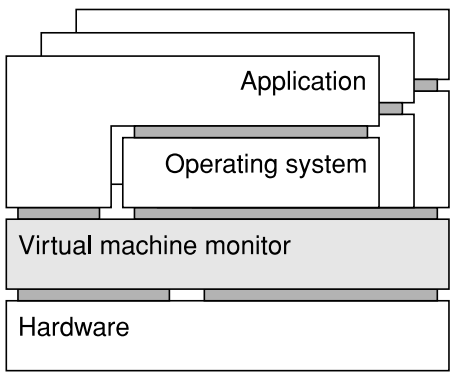
- Java Runtime Environment (JRE) $\rightsquigarrow$ Interpretation
- Windows-Anwendungen unter Linux $\rightsquigarrow$ Emulation

Arten von Virtualisierung

Welche Arten von Virtualisierung gibt es?

1. *Process Virtual Machine*
2. *Virtual Machine Monitor (VMM)*
3. *Containerization*

Virtual Machine Monitor



- Schicht zur kompletten Abschirmung der Hardware
- Mehrere unterschiedliche Betriebssysteme möglich

Beispiele:

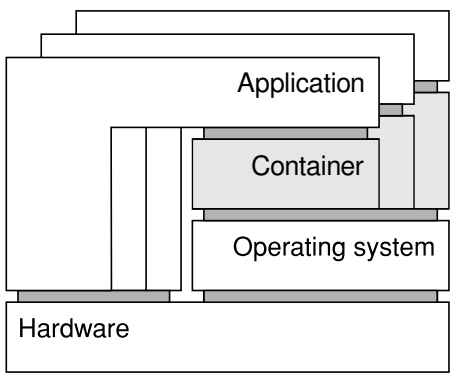
- VMware vSphere
- Microsoft Hyper-V
- Linux Kernel-based Virtual Machine (KVM)

Arten von Virtualisierung

Welche Arten von Virtualisierung gibt es?

1. *Process Virtual Machine*
2. *Virtual Machine Monitor (VMM)*
3. *Containerization*

Containerization



- Vorhandenes Betriebssystem wird als isolierte Ausführungsumgebung (Container) bereitgestellt
- \+ Hohe Performance durch Ausführung von Maschineninstruktionen
- \+ Geringer Overhead durch Wiederverwendung des Betriebssystems

Beispiele:

- Docker, Kubernetes

Inhalt

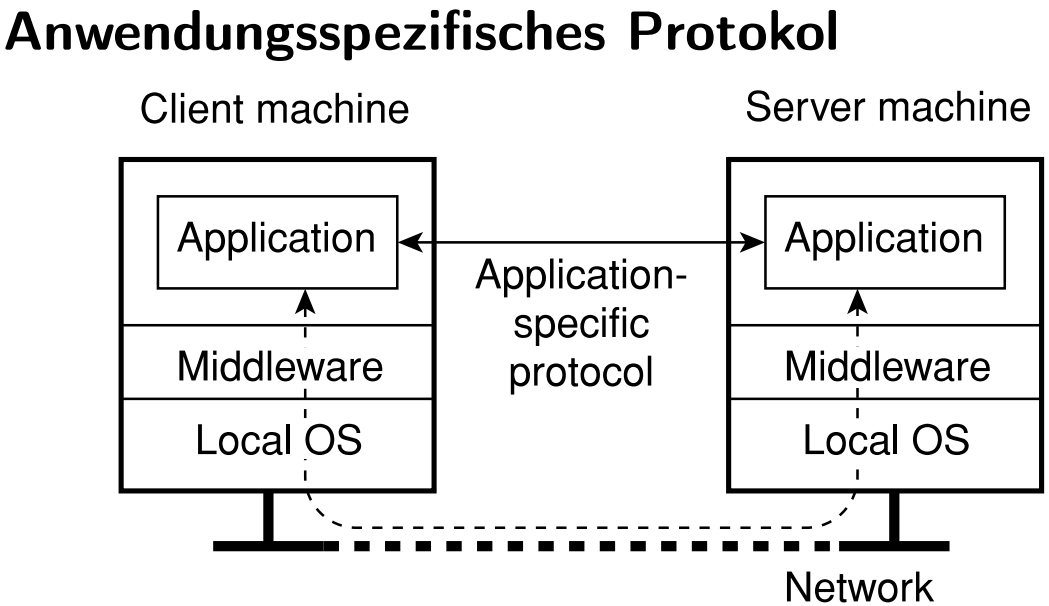
3. Prozesse
- Threads
- Virtualisierung
- Client-Realisierungen
- Server-Realisierungen
- Code-Migration

Client-Realisierungen

- Hauptaufgabe: Nutzerschnittstelle zur Interaktion mit einem Server
- Realisierungsarten: anwendungsspezifisch vs. anwendungsunabhängig

Client-Realisierungen

- Hauptaufgabe: Nutzerschnittstelle zur Interaktion mit einem Server
- Realisierungsarten: anwendungsspezifisch vs. anwendungsunabhängig

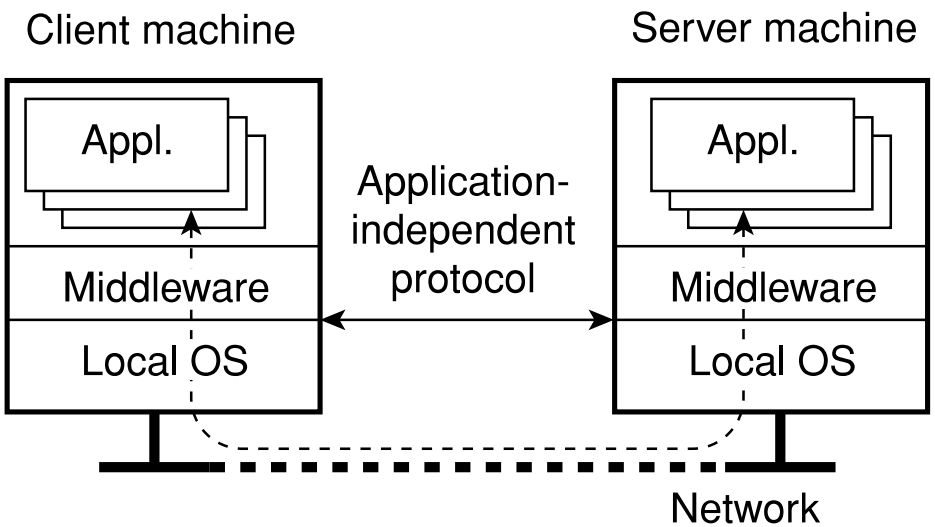


- Beispiel: Terminplaner via Kalender-Server

Client-Realisierungen

- Hauptaufgabe: Nutzerschnittstelle zur Interaktion mit einem Server
- Realisierungsarten: anwendungsspezifisch vs. anwendungsunabhängig

Anwendungsunabhängiges Protokol



- Beispiele: X Window System, Smartphone Apps

Client-seitige Unterstützung für Verteilungstransparenz

Zugriffstransparenz

- Nutzung von Client-Stubs (Platzhalter) von Server-Schnittstellen
- Stubs verbergen Kommunikation und Server-Unterschiede

Verortungs- und Ortsveränderungstransparenz

- Nutzung von ortsunabhängigen Namen bzw. Adressen
- Verbergen von Server-Ort und -Ortsveränderung innerhalb der Middleware

Replikationstransparenz

- Weiterleitung von Anfragen an replizierte Server
- Reduktion von replizierte Antworten werden zu Einzelantworten

Störungstransparenz

- Maskierung von Kommunikationsstörungen innerhalb der Middleware, z. B. durch wiederholte Verbindungsaufnahme oder Caching

Inhalt

3. Prozesse
- Threads
- Virtualisierung
- Client-Realisierungen
- Server-Realisierungen
- Code-Migration

Allgemeine Ansätze zur Server-Realisierung

Wie sieht der generelle Ablauf innerhalb eines Servers aus?

Allgemeine Ansätze zur Server-Realisierung

Wie sieht der generelle Ablauf innerhalb eines Servers aus?

1. Warte auf eine eingehende Anfrage von Clients
2. Bearbeite die Anfrage und sende eine Antwort
3. Setze das Warten auf Anfragen fort

Iterativer Server

- Eingehende Anfragen werden vom Server selbst behandelt
- Antwort wird vom Server zurück an den Client gesendet

Nebenläufiger Server

- Eingehende Anfragen werden an separate Threads/Prozesse weitergegeben

  ⇝ Server kann sofort auf weitere Anfragen warten
- Antwort wird vom verantwortlichen Thread/Prozess gesendet
- Beispiele:
  - Multithreaded Server ⇝ erzeugt neuen Thread (oder nutzt vorhandenen)
  - *Fork* Server (UNIX/Linux) ⇝ spaltet neuen Prozess via `fork` Systemaufruf ab

Demo: Iterativer und nebenläufige Server in Python

Gemeinsamer Teil: `common.py`

```
HOST = "127.0.0.1"
PORT = 12345

def handle_client(conn, addr):
    print(f"Verbindung von {addr}")
    while True:
        data = conn.recv(1024)
        if not data:
            break
        print(f"Empfangen: {data.decode()}")
    conn.close()
    print(f"Verbindung von {addr} geschlossen")
```

Demo: Iterativer und nebenläufige Server in Python

Iterativer Server: `iterative.py`

```
from socket import socket, AF_INET, SOCK_STREAM
from common import HOST, PORT, handle_client

with socket(AF_INET, SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen(1)
    print(f"Server läuft auf {HOST}:{PORT}")
    while True:
        try:
            conn, addr = server_socket.accept()
            handle_client(conn, addr)
        except:
            print(f"Server unterbrochen")
            break
```

Demo: Iterativer und nebenläufige Server in Python

Nebenläufiger Server mit Threads: `concurrent_threads.py`

```python
from socket import socket, AF_INET, SOCK_STREAM
from common import HOST, PORT, handle_client
from threading import Thread

with socket(AF_INET, SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server läuft auf {HOST}:{PORT}")
    while True:
        try:
            conn, addr = server_socket.accept()
            client_thread = Thread(target=handle_client, args=(conn, addr))
            client_thread.daemon = True
            client_thread.start()
        except:
            print(f"Server unterbrochen")
            break
```

Demo: Iterativer und nebenläufige Server in Python

Nebenläufiger Server mit Prozessen: `concurrent_fork.py`

```python
from socket import socket, AF_INET, SOCK_STREAM
from common import HOST, PORT, handle_client
import os

with socket(AF_INET, SOCK_STREAM) as server_socket:
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Server läuft auf {HOST}:{PORT}")
    while True:
        try:
            conn, addr = server_socket.accept()
            pid = os.fork()
            if pid == 0:  # Child process
                os.setsid()  # Detach from parent
                server_socket.close()  # Close child copy of server socket
                handle_client(conn, addr)
                os._exit(0)  # Exit child process when done
            else:
                conn.close() # Close server copy of client socket
        except:
            print(f"Server unterbrochen")
            break
```

Server-Kontaktpunkte

- Server „hört“ (*listen*) an einem Kontaktpunkt $\rightsquigarrow$ *Endpoint* oder *Port*
- Clients senden Anfragen an Server-Adresse & -Kontaktpunkt

Woher kennt ein Client die Server-Adresse?

Server-Kontaktpunkte

- Server „hört“ (*listen*) an einem Kontaktpunkt $\rightsquigarrow$ *Endpoint* oder *Port*
- Clients senden Anfragen an Server-Adresse & -Kontaktpunkt

Woher kennt ein Client die Server-Adresse?

- Feste Adressen, z. B. Nameserver 1.1.1.1 (Cloudflare) oder 8.8.8.8 (Google)
- Dynamische Ermittlung, z. B. Domain Name System (DNS)

Server-Kontaktpunkte

- Server „hört“ (*listen*) an einem Kontaktpunkt $\rightsquigarrow$ *Endpoint* oder *Port*
- Clients senden Anfragen an Server-Adresse & -Kontaktpunkt

Woher kennt ein Client die Server-Adresse?

- Feste Adressen, z. B. Nameserver 1.1.1.1 (Cloudflare) oder 8.8.8.8 (Google)
- Dynamische Ermittlung, z. B. Domain Name System (DNS)

Woher kennt ein Client den Server-Kontaktpunkt?

Server-Kontaktpunkte

- Server „hört“ (*listen*) an einem Kontaktpunkt $\rightsquigarrow$ *Endpoint* oder *Port*
- Clients senden Anfragen an Server-Adresse & -Kontaktpunkt

Woher kennt ein Client die Server-Adresse?

- Feste Adressen, z. B. Nameserver 1.1.1.1 (Cloudflare) oder 8.8.8.8 (Google)
- Dynamische Ermittlung, z. B. Domain Name System (DNS)

Woher kennt ein Client den Server-Kontaktpunkt?

- Statische Zuweisung, z. B.
  - HTTP $\rightsquigarrow$ TCP-Port 80, HTTPS $\rightsquigarrow$ TCP-Port 443
  - SSH $\rightsquigarrow$ TCP-Port 22, DNS $\rightsquigarrow$ UDP-Port 53
- Dynamische Ermittlung, z. B. via Endpoint-Daemon oder Superserver

Statische vs. dynamische Server-Kontaktpunkte

Statische Kontaktpunkte

- \+ Direkt nutzbar $\rightsquigarrow$ effizient und schnell
- – Begrenzte Anzahl und Standardisierbarkeit

Dynamische Kontaktpunkte

- – Nur indirekt nutzbar $\rightsquigarrow$ höhere Antwortzeiten
- \+ Flexible Nutzung für beliebige Dienste

Statische vs. dynamische Server-Kontaktpunkte

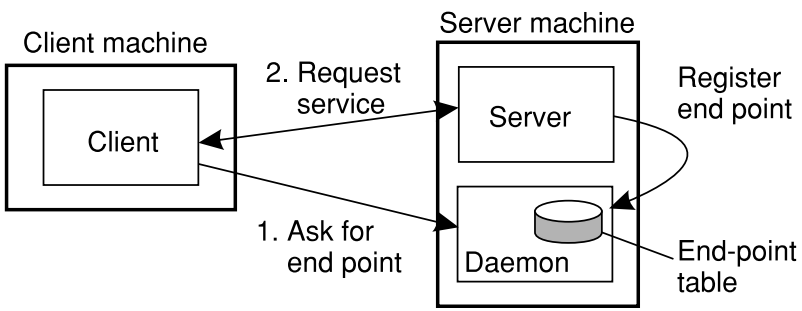
Statische Kontaktpunkte

+ Direkt nutzbar ⇝ effizient und schnell
– Begrenzte Anzahl und Standardisierbarkeit

Dynamische Kontaktpunkte

– Nur indirekt nutzbar ⇝ höhere Antwortzeiten
+ Flexible Nutzung für beliebige Dienste

Realisierungsoption: Endpoint-Daemon



- Separater Server mit festem Port und Verzeichnis mit Kontaktpunkten
- Server registrieren sich am Daemon
- Clients fragen Daemon nach spezifischen Server-Kontaktpunkten

– Jeder Server muss separat implementiert und ausgeführt werden

Statische vs. dynamische Server-Kontaktpunkte

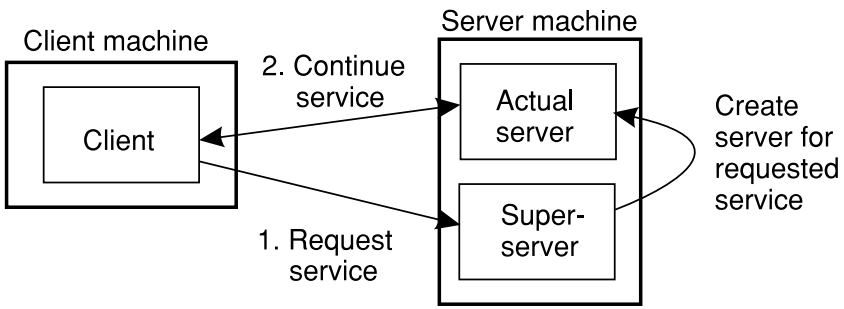
Statische Kontaktpunkte

+ Direkt nutzbar ⇝ effizient und schnell
– Begrenzte Anzahl und Standardisierbarkeit

Dynamische Kontaktpunkte

– Nur indirekt nutzbar ⇝ höhere Antwortzeiten
+ Flexible Nutzung für beliebige Dienste

Realisierungsoption: Superserver



- Separater Server der auf mehreren Kontaktpunkten hört
- Superserver startet spezifische Server wenn Anfragen eintreffen
- Beispiel: inetd unter UNIX/Linux

+ Nur ein Server wartet auf eingehende Anfragen ⇝ ressourceneffizient

Server und Zustände

Zustandsloser Server (*stateless*)

- Speichert keine Informationen über den Zustand der Clients
- Ändert eigenen Zustand unabhängig, d. h. ohne Clients zu informieren
- Verluste von Zustandsinformationen stören den Dienst nicht
- Sonderfall: Soft State $\rightsquigarrow$ Zustand wird zeitlich begrenzt gehalten

Was ist ein Beispiel für einen zustandslosen Server?

Server und Zustände

Zustandsloser Server (*stateless*)

- Speichert keine Informationen über den Zustand der Clients
- Ändert eigenen Zustand unabhängig, d. h. ohne Clients zu informieren
- Verluste von Zustandsinformationen stören den Dienst nicht
- Sonderfall: Soft State $\rightsquigarrow$ Zustand wird zeitlich begrenzt gehalten

Was ist ein Beispiel für einen zustandslosen Server?

Beispiel: Webserver

- antwortet auf HTTP-Anfragen und vergisst danach den Client
- Inhalte können sich ändern ohne dass Clients informiert werden
- benutzt Zustandsinformationen, z. B. Log-Daten, aber nicht zwingend

Server und Zustände

Zustandbehafteter Server (*stateful*)

- Speichert persistent Informationen über Clients

⇝ Informationen müssen explizit gelöscht werden

- Nach einem Absturz muss der Zustand wiederhergestellt werden
- Vorteil gegenüber zustandslosen Servern ⇝ Performance

Was ist ein Beispiel für einen zustandbehafteten Server?

Server und Zustände

Zustandbehafteter Server (*stateful*)

- Speichert persistent Informationen über Clients
- ⇝ Informationen müssen explizit gelöscht werden
- Nach einem Absturz muss der Zustand wiederhergestellt werden
- Vorteil gegenüber zustandslosen Servern ⇝ Performance

Was ist ein Beispiel für einen zustandbehafteten Server?

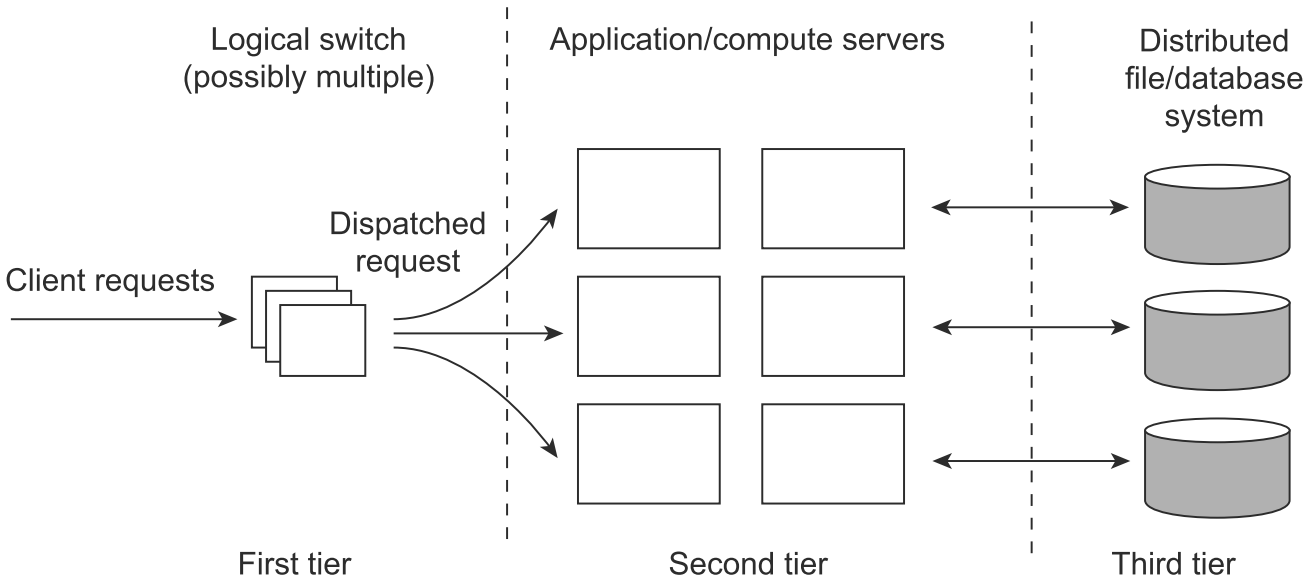
Beispiel: Dateiserver

- enthält Zustand welcher Client eine Datei aktuell liest/schreibt
- \+ erhöht Performance bei Lese-/Schreiboperationen
- − nach Absturz muss Zustand wiederhergestellt werden, z. B. um zu ermitteln welcher Client den aktuellen Stand einer Datei besitzt

Server-Cluster

- Menge von Computern in Netzwerk $\rightsquigarrow$ jeder führt einen oder mehrere Server aus

Beispiel: Dreistuffiger Server-Cluster



1. (Logischer) Switch zum Routing von Client-Anfragen
2. Dedizierte oder geteilte Anwendungs- bzw. Rechenserver
3. Datenverarbeitungsserver, z. B. Datei- oder Datenbankserver

Management von Server-Clustern

- Einfach: Administratoren loggen sich auf einzelnen Knoten ein
- Fortgeschritten: Zentrale Verwaltungsoberflächen für kollektives Management

⇝ Klassische Methoden für sehr große Cluster (Tausende Knoten) ungeeignet

- Mit wachsender Knotenzahl steigt die Wahrscheinlichkeit von Fehlern
- Beispiel: $N = 1000$ Server, Fehlerrate $p = 0,1\%$

$$P(\text{kein Server fällt aus}) = (1-p)^N \approx 36\,\%$$

Heutige Realität

- Automatisierung, Container-Orchestrierung und serverlose Architekturen ⇝ effizientes und dynamisches Management
- Self-healing-Systeme ⇝ autonome Fehlererkennung und Reparatur
- Hybrid- und Multi-Cloud-Umgebungen ⇝ flexible und skalierbare Cluster-Verwaltung über verschiedene Plattformen hinweg
- Fernverwaltungstools sichern kontinuierlichen Betrieb und Sicherheit

Inhalt

3. Prozesse
- Threads
- Virtualisierung
- Client-Realisierungen
- Server-Realisierungen
- Code-Migration

Code-Migration

- Bisher wurde Kommunikation eingesetzt um Daten auszutauschen
- Code-Migration ⇝ Verschieben von Programmen, z. T. während der Ausführung

Gründe

- Verbesserung der Leistung ⇝ Prozesse von stark auf weniger ausgelastete Maschinen verschieben
- Minimierung von Kommunikation ⇝ Code an den Ort der Daten bringen, z. B. Client zu Server oder umgekehrt
- Flexibilität durch Code-Verschiebung
  - ⇝ Software kann bei Bedarf zum Client geladen werden (anstatt vorab installiert)
  - ⇝ Verteilte Systeme können zur Laufzeit dynamisch konfiguriert werden

Herausforderungen

- Sicherheit ⇝ Vertrauen in dynamisch geladene Programme ist kritisch
- Standardisierte Protokolle für Download und Ausführung erforderlich

Modelle für Code-Migration

Schwache Mobilität

- Nur Programm-Code und ggf. Initialisierungsdaten werden migriert

⇝ Programm startet von vordefinierten Startpunkten

Starke Mobilität

- Ausführungssegment (z.B. Instruktionszeiger und Stack) wird mit migriert

⇝ Laufender Prozess kann gestoppt, verschoben und fortgesetzt werden

Sender-initiierte Migration

- Computer der den Code aktuell enthält initiiert Migration
- Beispiel: Upload von Programmen zu einem Rechenserver

Empfänger-initiierte Migration

- Computer der den Code erhalten möchte initiiert Migration
- Beispiel: Download und Ausführung von Java-Applets

Weitere Herausforderungen der Code-Migration

Zugriff auf Ressourcen

- Programm-Code und Daten können zumeist unverändert migriert werden

⇝ Ressourcennutzung, z. B. Netzwerkports oder Dateizugriffe, muss im Zuge der Migration angepasst werden

Migration in heterogenen Systemen

- Systeme bestehend aus verschiedenen Betriebssystemen und Architekturen
- Lösungsansätze
  - Nutzung von Virtuellen Maschinen, z. B. Java Virtual Maschine (JVM)
  - Migration von Ausführungsumgebungen anstatt einzelner Prozesse
    ⇝ Prozesseigenschaften und Ressourcenbindungen bleiben direkt erhalten

Migration von Betriebssystemumgebungen

- Nutzung von Virtualisierung mittels *Virtual Machine Monitor* (VMM)
- Beispiel: Live-Migration von VMs innerhalb eines VMware-Server-Clusters

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz